Hello Java!

Let's Code Together

by Aditya Varma

Preface

Index

Introduction

//Introduction to Java

What is Java?

Java is a high-level, general-purpose programming language used to build everything from small applications to massive enterprise systems. You can think of Java as a universal translator, you write code once, and it automatically adapts to different devices and operating systems.

Java belongs to the category of compiled languages.
This means:

- You write code in Java.
- A compiler converts it into a special format called bytecode.
- This bytecode is then executed by the Java Virtual Machine (JVM) on any device.

You can imagine Java's compiler as a chef who prepares a basic dish (bytecode), and the JVM is like local cooks in each country adding their own flair depending on their kitchen (OS/platform).
The recipe stays the same, only the cooking varies.

Why is bytecode important?

Because bytecode is not tied to any specific device. It's a "universal" format that works everywhere, that's where Java gets its magic.

Features of Java!

1. Platform Independence

Java's famous idea is "Write Once, Run Anywhere" (WORA).

This means:

- You write Java code once.
- It gets converted into bytecode.
- That bytecode runs on any system where a JVM exists — Windows, Linux, macOS, Android, etc.

Think of bytecode like a movie subtitle file, the same subtitle file works on many different media players. Each player interprets it in its own way, but you don't need to create separate subtitles for each one. This reduces work for programmers, no need to maintain different code versions for each platform.

2. OOP Language (Object-Oriented Programming)

Java uses the object-oriented approach, which means it organizes programs into "objects", mini units that combine data and functions.

You can imagine objects like real-world things:

- A Car object has data (color, model) and actions (start, brake).
- A Student object has data (name, age) and actions (study, takeExam).

OOP helps keep programs clean, organized, and easier to expand.

3. Secure & Robust

Java takes security seriously.

It avoids many common programming hazards because:

- Memory management is automatic
- Many risky operations are restricted
- Programs run inside the JVM "sandbox," giving an extra layer of protection

Java also does automatic memory handling:

- When you create an object, Java automatically allocates memory for it.
- When the object is no longer used, the Garbage Collector recycles that memory.

You can think of it like a house where a cleaning robot (GC) quietly removes stuff you no longer need, preventing a messy room (memory leak).

This makes Java stable (robust) and safe.

4. Portable

Java programs can move from one computer to another without modification.

Since the bytecode stays the same everywhere, Java apps don't care whether they're running on:

- A laptop
- A server
- A mobile device
- A smart TV

It's as portable as carrying a USB drive with universal files plug in anywhere, and it works.

What Java is Used For?

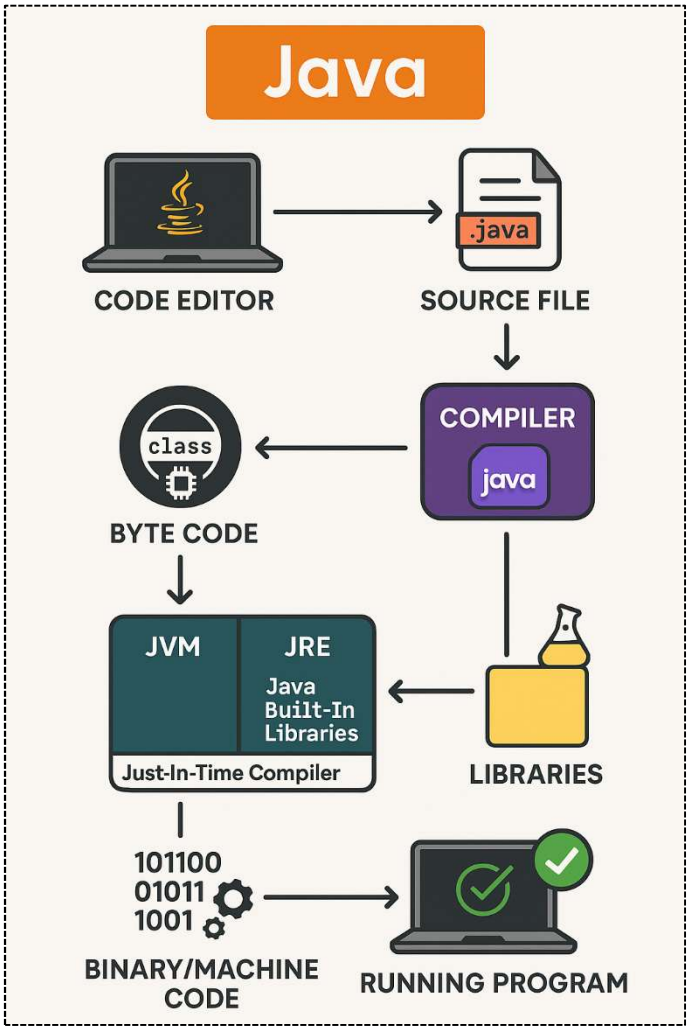
Java has been around for decades and still powers a huge part of the software world. It's used in:

- 💼 Enterprise software (banking systems, ERPs, corporate tools)
- 📱 Android apps (Java was the primary Android language for years)
- 🌐 Web applications (back-end servers using Java)
- ☁️ Cloud-based systems
- 🎮 Games (lightweight games and certain engines)
- 🖥️ Desktop applications

- 🔌 Embedded systems (smart cards, sensors, appliances)

Basically, Java is like a Swiss Army Knife — you'll find it almost everywhere in the tech world.

Write Once, Run Anywhere



//JVM, JRE & JDK – The Three Pillars of Java

When learning Java, these three names show up everywhere. They sound technical, but once you see them as parts of a mini Java ecosystem, everything becomes easy to understand.

Let's break them down with a simple analogy:

1. JVM – Java Virtual Machine

Think of the JVM as a translator who understands bytecode.

- You give JVM the bytecode produced by the compiler.
- JVM reads it, understands it, and turns it into instructions your computer can execute.
- Each operating system has its own version of the JVM, but they all understand the same bytecode.

Simple Analogy:

Imagine you write a letter in a universal language. The JVM is like a local interpreter who reads the letter and speaks in the local language (Windows, Linux, Mac). That's how Java achieves Write Once, Run Anywhere.

2. JRE – Java Runtime Environment

The JRE is the "playground" where Java programs actually run.

It includes:

- The JVM
- Built-in Java libraries (pre-written code you can use)
- Tools needed to run programs

But note: You cannot write or compile Java code with the JRE. It's only for running applications.

Analogy:

If JVM is the performer, then JRE is the stage + lights + tools that allow the performer to do their job.

3. JDK – Java Development Kit

The JDK is the complete toolset for people who want to write Java programs.

It contains:

- Everything inside the JRE
- Plus, tools for developers
    - Java compiler (javac)
    - Debuggers
    - Documentation tools

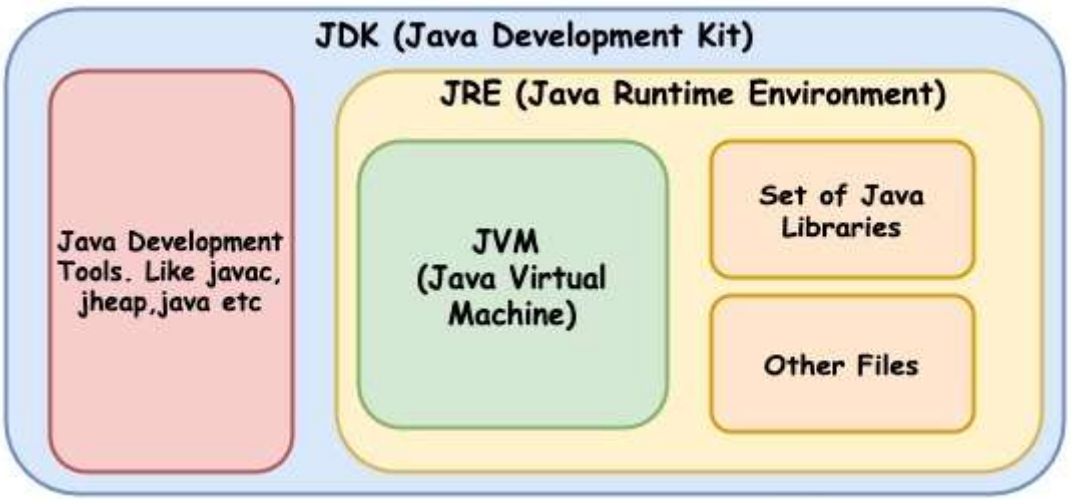
Analogy:

If JRE is the stage, then JDK is the entire theatre building, including:

- Stage (JRE)
- Backstage tools

- Equipment
- Workshops

JDK = Everything you need to create, compile, and run Java code.



//Installing Java (JDK)

Check if Java is already installed:

On Windows:

1. Open Command Prompt (CMD)
2. Type:

```
java -version
```

If Java is installed, you will see something like:

```
java version "17.0.10"
```

If Java is NOT installed, you will see:

```
'java' is not recognized as an internal or external command
```

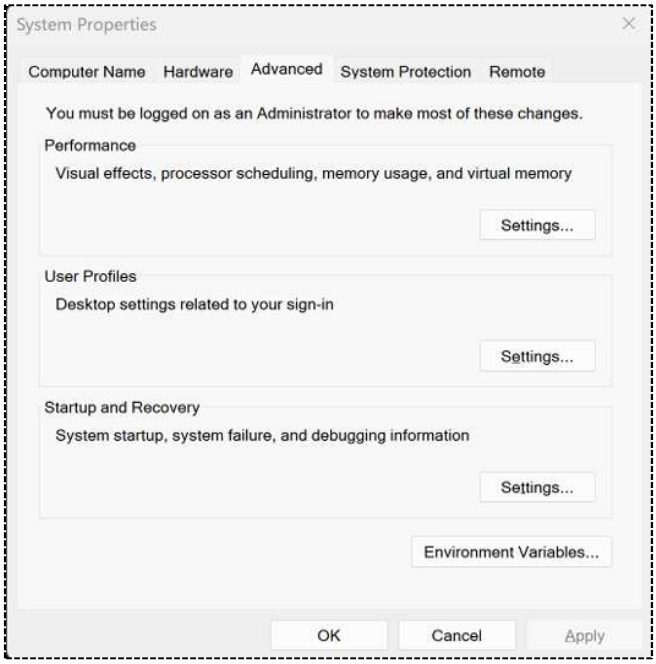
Installing Java (JDK):

1. Visit the Oracle Java SE Downloads page at https://www.oracle.com/java/technologies/downloads/.
2. Select the appropriate installer for your Windows operating system (for example, Windows x64 Installer).
3. Download the installer by clicking on the file link.
4. Run the downloaded installer (the .exe file) and follow the on-screen instructions to complete the installation.

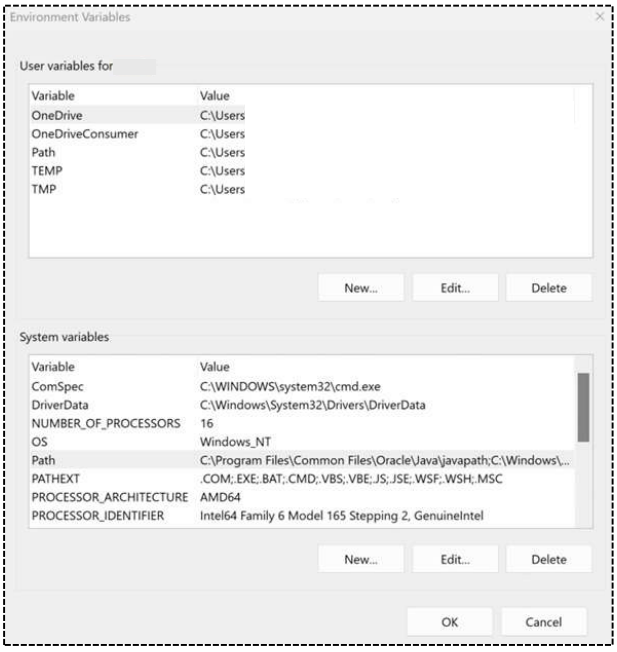
Setting Up Environment Variables:

(This step is sometimes done automatically by newer installers, but you should know it.)

1. Find the JDK installation path:
   Example path:
   `C:\Program Files\Java\jdk-25`
2. Add JAVA_HOME:
   - Search “Edit the system environment variables” -> Click Environment Variables
   - Under System variables, click New
   - Variable name: JAVA_HOME
   - Variable value: (your JDK folder path): C:\Program Files\Java\jdk-25



3. Add JDK to PATH:
    - Go to System variables → Path → Edit
    - Click New, add: `%JAVA_HOME%\bin`



4. Verify installation:
    - Open CMD and type:

```
java -version
```

You should see version numbers.

//Installing an IDE (VS Code / IntelliJ)

Option A: VS Code

1. Download and install VS Code
2. Open VS Code → Go to Extensions
3. Install:
    - Extension Pack for Java (important)
    - Debugger for Java
    - Java Test Runner (optional)

That's it VS Code is ready for Java development.

Option B: IntelliJ IDEA

(More beginner-friendly than VS Code. Handles everything automatically.)

1. Download IntelliJ IDEA Community Edition (free)
2. Install and open it
3. IntelliJ automatically detects your JDK
    - If it asks: just select the JDK folder (example: C:\Program Files\Java\jdk-17)
4. Create a New Java Project and start coding.

//Creating & Running Your First Java Program

Using CMD (Command Prompt):

Step 1: Create a new file named Hello.java in notepad.

Java requires the filename and class name to be the same.

Step 2: Write this code in the notepad,

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

Step 3: Save the file in a folder you want.

Step 4: Open CMD and go to that folder.

```
cd C:\JavaPrograms
```

Step 5: Compile the file

```
javac Hello.java
```

This will create:

Hello.class (Your bytecode file)

Step 6: Run the program

```
java Hello
```

Output:

```
Hello World
```

Using IntelliJ IDEA (recommended):

Step 1: Open IntelliJ → Create New Project

- Select New Project
- Choose Java
- Select JDK 17
- Click Create

Step 2: Create a Java File

Inside src folder:

1. Right-click src
2. Select New → Java Class
3. Name it:

Step 3: Write the Program

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

Step 4: Run the Program

At the top-left of the editor you will see a green play button ▶. You will get the Output.

//Basic Structure of a Java Program

Every Java program follows a specific structure. Once you understand this structure, Java becomes much easier to read and write.
Let's break it down part by part.

Understanding Class Structure:

Every Java file contains **at least one class**.

General Structure:

```
public class ClassName {
    // variables (optional)
    // methods (optional)
}
```

Points to remember:

- The class name must match the file name
  e.g., file: Hello.java → class: Hello
- A class is like a container that holds your code.
- Everything in Java lives inside a class.

Example: Think of a class as a **house**, and all your code sits inside it.

The main() Method Explained:

This is the starting point of **every** Java program.

```
public static void main(String[] args) {
    // program starts here
}
```

What each word means:

- public → Anyone can run this method
- static → Runs without creating an object
- void → Does not return anything
- main → Special method where program starts
- String[] args → Accepts inputs from the user (command-line arguments)

Think of main() as:

The **front door** of your program. Java starts running your code by entering through this door.

Compiling & Running a Java Program:

Compilation: Java code (Hello.java) → compiled into bytecode → Hello.class

javac Hello.java

Execution: Run the bytecode using the JVM

java Hello

Common Errors Beginners Face:

1. File name does not match class name:

   public class Hello {}

   but file is named: HelloWorld.java

   Fix: Rename file to Hello.java

2. Missing semicolon
3. Wrong capitalization:

   Java is **case-sensitive**.

   System ≠ system
   main ≠ Main
   String ≠ string

   Fix: Use exact uppercase/lowercase.
4. Missing curly braces { }:

   Every opening brace { must have a closing brace }.
5. Running the program with .java extension:

   You never include the .java when running.
6. Typing code outside the class:

   Everything must be **inside the class** and mostly inside the **main method** for beginners.

Quick Visual Summary:

```
public class Hello {        ← Class begins
    public static void main(String[] args) {   ← Main method begins
        System.out.println("Hello World");     ← Statement
    }                                          ← Main closes
}                                              ← Class closes
```

Java Basics

Lesson Program: Tokens in Java.

Source Code:

Tokens.java

```java
// Program: Tokens in Java

public class Tokens {
    public static void main(String[] args) {

        System.out.println("\n=== TOKENS IN JAVA ===");

        System.out.println("""
Java programs are made of tokens:
1. Keywords
2. Identifiers
3. Literals
4. Symbols & Operators
""");

        System.out.println("Keyword Example: public, class, static");
        System.out.println("Identifier Example: TokensInJava, count, value");

        System.out.println("Literals:");
        System.out.println(10);
        System.out.println(3.14);
        System.out.println('A');
        System.out.println("Hello Java");
        System.out.println(true);

        System.out.println("Symbols & Operators:");
        System.out.println("Semicolon ; ends statements");
        System.out.println("Using + operator: " + ("Java" + " Tokens"));
    }
}
```

Lesson Program: Comments in Java.

Source Code:

Comments.java

```java
// Program: Comments in Java

public class Comments {
    public static void main(String[] args) {

        System.out.println("\n=== COMMENTS IN JAVA ===");

        System.out.println("""
Types of comments:
1. Single-line
2. Multi-line
3. Documentation comments
""");

        // Single-line comment example
        System.out.println("Single-line comment example printed.");

        /*
           This is a multi-line comment.
           Used for long explanations.
        */
        System.out.println("Multi-line comment example printed.");

        /**
         * Documentation comments help create official docs.
         */
        System.out.println("Documentation comment example printed.");
    }
}
```